Architecture Specification

MedInfo Assistant - Medical Research & Report Analysis Multi-Agent GenAI Platform

Generated 2026-10-15 from 03-ARCHITECTURE.md - information only, not medical advice.

Contents

0. Document control
1. Architectural drivers
2. Architectural style
3. Logical view
4. Process view (runtime and concurrency)
5. Development view (code organisation and dependency rules)
6. Physical view (deployment)
7. Scenarios (+1 view): quality-attribute scenarios
8. Safety architecture (defence in depth)
9. Patterns used
10. Cross-cutting concerns
11. Technical debt and evolution

0. Document control

Field	Value
Document	Architecture Specification (4+1 views)
Version	2.0 (production-grade)
Status	Draft for approval
Date	2026-10-14
Inputs	PRD, HLD
Detailed by	LLD, ADRs

1. Architectural drivers

1.1 Business and safety drivers

Driver	Source	Architectural response
Never diagnose or prescribe	PRD G3	Scope agent + verifier + code-built templates on every path
Emergency first	PRD G2	Triage before any other step; notice built in code; outage-proof
Evidence or nothing	PRD G1	Retrieval-gated generation; verifier fails closed
PHI protection	PRD G4	Local redaction before egress; encrypted memory; owner scoping
Auditable decisions	Governance	Fixed pipeline order; structured outputs; audit events

1.2 Quality-attribute drivers (ranked)

1. **Safety** (no unsafe output, emergency never suppressed)
2. **Privacy/security**
3. **Correctness/traceability** of answers
4. **Availability/resilience** under provider failure
5. **Maintainability/testability** (every external dependency fakeable)
6. **Performance** (acceptable interactive latency with streamed progress)

1.3 Constraints

Python 3.11 + FastAPI; OpenAI-compatible providers only; Supabase Postgres + pgvector; Redis; Docker, Kubernetes (EKS), Terraform; synthetic data until approvals.

---

2. Architectural style

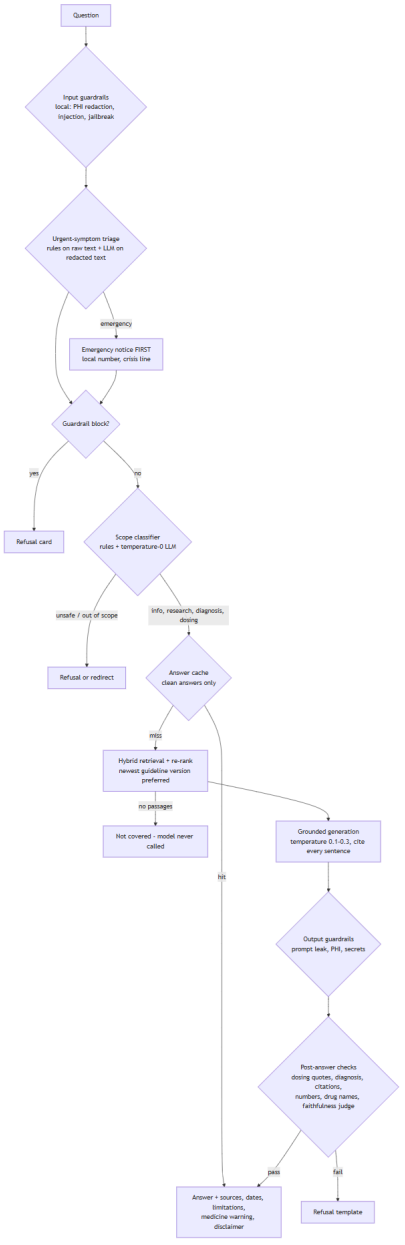
A **layered, ports-and-adapters backend** hosting a **fixed multi-agent pipeline with a deterministic safety spine**, behind a **static frontend + reverse proxy**.

- Agents are narrow and specialised; they do not negotiate freely. `ChatService` defines the order; LangGraph executes the RAG graph (rewrite -> retrieve -> generate) and the research tool loop.
- Deterministic components (rules, guardrails, checks) bracket every probabilistic component (LLM).
- This trades open-ended autonomy for **predictability and auditability**, the right trade-off for medical information.

---

3. Logical view

3.1 Agents and safety components



#	Component	Kind	Input -> Output	May not
1	Input guardrails	Deterministic	raw question -> redacted question / block	Pass PHI to any model
2	Triage agent	Rules + LLM (T=0)	raw (rules) + redacted (LLM) -> emergency/crisis decision	Write the emergency text
3	Scope agent	Rules + LLM (T=0)	redacted question -> 1 of 7 classes	Relax a rules decision
4	Query rewriter	LLM (T=0)	follow-up + history -> standalone queries	Influence the answer text

#	Component	Kind	Input -> Output	May not
5	Retriever	Deterministic + DB	queries -> fused candidates (RRF)	Return another user's uploads
6	Re-ranker	LLM (T=0)	candidates -> chosen passages	Add content; fails open
7	Research agent	ReAct tool loop (T=0)	question -> corpus/web passages	Write the answer; leave the allow-list
8	Answer agent	LLM (T=0.1-0.3)	passages -> cited sentences + limitations	Run without passages
9	Output guardrails	Deterministic	answer -> redacted / blocked	Let prompt leaks through
10	Verifier	Deterministic + judge (T=0)	answer + passages -> pass / refuse	Pass unverifiable claims (fails closed)
11	Templates	Code	outcome -> fixed safety wording	Be altered by any model

Each of components 1-10 that runs records a step in an **agent trace** (`rag/trace.py: AgentTrace` ; retriever and re-ranker share one step, plus an answer-cache step and a final result step): agent, outcome, short detail, status (ok / warn / stop / skip) and duration in ms. `ChatService` attaches it to every answer as `safety.trace` , and the UI shows it as "How this answer was produced". The trace holds decisions and counts only (never question, answer or passage text, nor upload filenames). See LLD 4.1.4. Status: Implemented + tested offline.

3.2 Domain modules

Module	Responsibility
`safety`	Triage, scope, answer checks, templates, safety-suite runner
`guardrails`	Policy engine and policy file
`rag`	Use case (`service`), retrieval, ingestion, memory, cache, review, report, evaluation, prompts
`agents`	LangGraph graphs (RAG, research)
`core`	Cross-cutting: logging, metrics, audit, auth, circuit breaker, rate limiter, KV

3.3 Key interfaces (ports)

`ChatModel` , `Embedder` , `Retriever` , `Reranker` , `VectorIndex` , `DocumentRegistry` , `OcrEngine` , `DatabaseProbe` , `KeyValueStore` , `ConversationStore` , `ReviewStore` , `WebSearch` , `ToolModel` , `SigningKeyResolver` . Each has a production adapter and an offline fake.

---

4. Process view (runtime and concurrency)

Aspect	Design
Server	uvicorn, **one worker per container**; scale by replicas
Request handling	Sync FastAPI handlers run in the server thread pool; provider SDK calls are blocking and isolated per request
Streaming	`/api/chat/stream` : the pipeline runs in a worker thread (context copied for request ids); phases are pushed to a queue and emitted as SSE; nginx proxies unbuffered
Graph execution	LangGraph `stream(updates)` emits node completions -> phase events
Shared state	Only in Redis/Postgres (caches, counters, conversations); pods are stateless
Locks	Process-local locks only for lazily initialised resources (OCR engine, agent verification)
Background work	None in the API process (no disk writes); evaluation and purge jobs run outside the API container

5. Development view (code organisation and dependency rules)

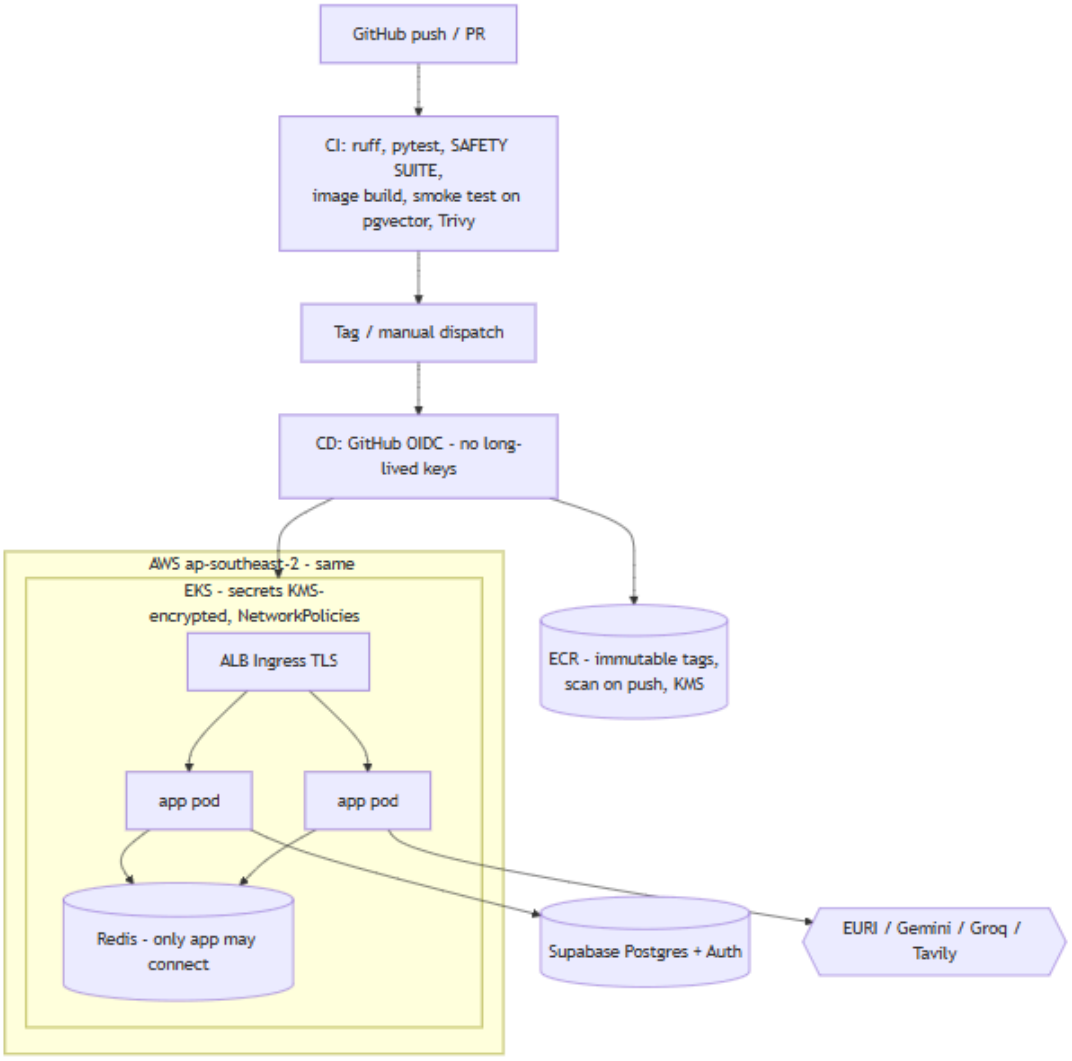
```
backend/app/
  api/         -> may import: container, core, rag.schemas            (HTTP only)
  rag/service  -> may import: safety, guardrails, rag.*, agents, core  (use case)
  agents/      -> may import: rag.interfaces, rag.prompts, core        (graphs)
  safety/, guardrails/, rag/* (domain) -> rag.interfaces, core         (no provider SDKs)
  rag/providers.py, container.py       -> the ONLY place provider SDKs are imported
  core/        -> no dependency on rag/safety/api
frontend/src/  -> static assets only; talks to /api
```

Enforced by review rules (`.claude/rules/engineering.md`) and by tests that run the whole pipeline with fakes (if a domain module imported an SDK, offline tests would need credentials).

---

6. Physical view (deployment)

Node	Runs	Network policy
ALB	TLS termination	Internet -> frontend only
Frontend pods (2-4)	nginx + static UI, `/api` proxy	Ingress from VPC/ALB; egress to backend + DNS
Backend pods (2-8)	FastAPI + agents	Ingress from frontend + monitoring; egress to Redis, DNS, 443/5432/6543
Redis pod	Cache/memory/limits	Ingress from backend only; no egress
Supabase (managed)	Postgres + pgvector, Auth	TLS; credentials from secrets
Providers (external)	Models, search	HTTPS egress


GitHub push / PR
CI: ruff, pytest, SAFETY SUITE,
image build, smoke test on pgvector, Trivy
Tag / manual dispatch
CD: GitHub OIDC - no long-lived keys
AWS ap-southeast-2 - same
EKS - secrets KMS-encrypted, NetworkPolicies
ALB Ingress TLS
app pod
app pod
ECR - immutable tags, scan on push, KMS
Redis - only app may connect
Supabase Postgres + Auth
EURI / Gemini / Groq / Tavily

7. Scenarios (+1 view): quality-attribute scenarios

#	Attribute	Stimulus	Expected response	Verified by
S1	Safety	"Crushing chest pain" while the model provider is down	Emergency notice returned first, no 5xx hiding it	`test_emergency_survives_provider_outage` ; live chest-pain run
S2	Safety	Model tries to output "Take 40 mg twice daily"	Answer replaced by refusal + pharmacist warning	Safety suite (adversarial model)
S3	Privacy	Question contains name, DOB, MRN, phone	None reach any model prompt, log, memory or embedding	`test_guardrails_and_phi.py`
S4	Security	Upload contains "IGNORE ALL PREVIOUS INSTRUCTIONS"	Line neutralised, chunk flagged, payload never in prompts	`test_uploaded_paper_injection_is_neutralised_and_flagged`
S5	Correctness	No relevant passage exists	"Not covered"; answer model not called	Live run (pancreatic cancer)
S6	Resilience	Primary model returns quota errors	Next model used; all quota -> 429 + Retry-After	Fallback tests
S7	Performance	User waits for an answer	Phases appear progressively via SSE	`test_stream_emits_real_phases_then_the_same_answer`
S8	Modifiability	Swap embedding model	New collection name; no mixed vectors; re-ingest	Collection naming in settings

#	Attribute	Stimulus	Expected response	Verified by
S9	Auth	Supabase token issued seconds "in the future" (clock skew)	Accepted within 30 s leeway; far-future rejected	Live 3-role check; skew test

8. Safety architecture (defence in depth)

Layer	Mechanism	Fails
L1 Input	Guardrail block/redact (local)	Closed (block)
L2 Triage	Rules OR LLM -> emergency first	Rules still apply if LLM fails
L3 Scope	Stricter of rules/LLM	Rules apply if LLM fails
L4 Evidence	Retrieval gate; untrusted labelling	No passages -> not covered
L5 Generation	Grounded prompt, fixed format, low temperature	Output still verified
L6 Output guard	Prompt-leak block, PHI/secret redaction	Closed (block)
L7 Verifier	Citation/number/drug/dose/diagnosis checks, judge	Closed (refuse)
L8 Presentation	Code-built emergency/disclaimer/limitations; text-only UI	N/A
L9 Monitoring	Safety metrics, alerts, clinician review	Human escalation

9. Patterns used

Pattern	Where	Purpose
Ports and adapters	`rag/interfaces.py`, `providers.py`	Swap providers; offline tests
Composition root	`container.py`	Single wiring point; test overrides
Pipeline / chain of responsibility	`ChatService._answer`	Ordered safety stages
Circuit breaker + fallback	`core/circuit_breaker.py`, `rag/fallback.py`	Provider resilience
Retry with exponential backoff	`with_retries`	Transient failures
Reciprocal rank fusion	`retrieval.reciprocal_rank_fusion`	Merge vector + keyword rankings
ReAct tool loop	`agents/research.py`	Evidence gathering
Cache-aside with versioned keys	`rag/cache.py`	Invalidation on corpus change
Gateway + routing table	`rag/gateway.py`, `llm_gateway.yaml`	One entry point for model calls: task routing, token limits, accounting (ADR-006)
Content-addressed storage	`chunking.chunk_ids`	Incremental upsert
Policy-as-data	`policy.yaml`, `red_flags.yaml`	Reviewable safety configuration

10. Cross-cutting concerns

Concern	Approach
Configuration	pydantic-settings; `.env` at repo root; production validation of critical settings
Logging	Text locally (with `extra=` fields), JSON in containers; request ids; no PHI
Errors	Domain `AppError` hierarchy mapped centrally to HTTP
Auth	JWKS-verified ES256/RS256; roles from `app_metadata` ; 30 s skew leeway
i18n	English only (planned)
Time	UTC everywhere; ISO-8601 in APIs

11. Technical debt and evolution

Item	Impact	Plan
Classifiers run sequentially	Latency	Run triage + scope in parallel
Single Redis instance	Cache loss on restart (tolerated)	Managed Redis with replica
Pattern-based PHI redaction	May miss unusual formats	Add NER-based detector behind the same policy stage
Quote-only dosing answers unreliable live	Over-refusal	Prompt/format tuning with live evaluation
Report format Markdown only	Usability	PDF export
Docker build not yet run locally	Delivery risk	Run in CI; fix findings